DATA WAREHOUSE DE VENDAS DO ADVENTUREWORKS: MODELAGEM ESTRELA E ETL INCREMENTAL ORQUESTRADO COM APACHE AIRFLOW

Lucas de Lima Nascimento
James Alves
Centro Universitário Salesiano - UniSales
Curso: Sistemas de Informação

RESUMO

Este artigo apresenta o projeto e a implementação de um *Data Warehouse* (DW) de vendas construído a partir do banco transacional AdventureWorks2016. O modelo multidimensional segue o padrão estrela, com uma tabela fato no grão de item de pedido e seis dimensões, implementado em PostgreSQL. O processo de extração, transformação e carga (ETL) foi desenvolvido em Python e orquestrado pelo Apache Airflow, com carga incremental baseada no *Change Tracking* do SQL Server: após a carga inicial, apenas pedidos e cadastros alterados são reprocessados, e os dados e o ponto de controle são confirmados na mesma transação. Dez indicadores de desempenho comercial foram implementados como *views* em *Structured Query Language* (SQL). A carga inicial gravou 121.317 itens de 31.465 pedidos; em um teste controlado, uma alteração na origem levou a execução incremental a ler somente os 12 itens do pedido afetado e regravar apenas o item modificado. Os indicadores apontam receita líquida de US$ 109,8 milhões, concentração de 86,2% da receita na categoria *Bikes* e forte oscilação mensal provocada pelo canal de revendedores.

Palavras-chave: *data warehouse*; modelo estrela; ETL incremental; Apache Airflow; AdventureWorks.

ABSTRACT

This paper presents the design and implementation of a sales data warehouse built from the AdventureWorks2016 transactional database. The multidimensional model follows the star schema, with a fact table at order-line grain and six dimensions, implemented in PostgreSQL. The extract, transform and load (ETL) process was written in Python and orchestrated with Apache Airflow, using incremental loading based on SQL Server Change Tracking: after the initial load, only changed orders and master data are reprocessed, and data and checkpoint are committed in the same transaction. Ten business indicators were implemented as SQL views. The initial load stored 121,317 lines from 31,465 orders; in a controlled test, a source change made the incremental run read only the 12 lines of the affected order and rewrite only the modified line. The indicators show net revenue of US$ 109.8 million, 86.2% of revenue in the Bikes category and strong monthly swings driven by the reseller channel.

Keywords: data warehouse; star schema; incremental ETL; Apache Airflow; AdventureWorks.

1 INTRODUÇÃO

Sistemas transacionais (*Online Transaction Processing* – OLTP) são projetados para registrar operações com rapidez e integridade, por isso armazenam dados em esquemas altamente normalizados. Essa organização é adequada ao registro de pedidos, mas dificulta consultas analíticas, que precisam percorrer muitas tabelas e grandes volumes históricos. O *Data Warehouse* resolve esse problema ao integrar os

dados em uma base orientada a assuntos, histórica e voltada à consulta (Inmon, 2005).

A modelagem dimensional proposta por Kimball e Ross (2013) organiza o DW em tabelas fato, que guardam medidas numéricas de um processo de negócio, e tabelas de dimensão, que guardam o contexto descritivo dessas medidas. No modelo estrela, cada dimensão se liga diretamente à fato, o que simplifica as consultas e melhora o desempenho de agregações (Star..., 2026).

O objetivo deste trabalho é projetar e implementar um DW de vendas a partir do banco de exemplo AdventureWorks2016 (Microsoft, 2026a), com: análise do modelo OLTP; definição de dez indicadores; modelo estrela em PostgreSQL; ETL incremental orquestrado pelo Apache Airflow; e cálculo dos indicadores em SQL. O código-fonte completo (DAG, scripts do DW, scripts dos indicadores e documentação) está disponível em https://github.com/lucaslnasc/adventureworks-dw (Adventureworks-dw..., 2026). O texto segue as normas do guia institucional (Centro Universitário Salesiano, 2024).

2 DESENVOLVIMENTO

O ambiente foi montado com Docker Compose e reúne quatro serviços: SQL Server 2022 com o AdventureWorks2016 restaurado (origem), PostgreSQL 17 (DW) (PostgreSQL Global Development Group, 2026), Apache Airflow 3.3.2 (orquestração) e um PostgreSQL separado para os metadados do Airflow. O ETL foi escrito em Python, com os drivers pymssql e psycopg.

2.1 ANÁLISE DO MODELO TRANSACIONAL

O AdventureWorks2016 representa uma fabricante fictícia de bicicletas e possui cerca de 70 tabelas distribuídas nos esquemas *Person*, *Sales*, *Production*, *Purchasing* e *HumanResources*, em terceira forma normal. O processo de negócio escolhido foi **vendas**, por concentrar as informações de receita, clientes e produtos. Ele envolve dez tabelas: SalesOrderHeader (cabeçalho, 31.465 pedidos), SalesOrderDetail (itens, 121.317 linhas), Product, ProductSubcategory, ProductCategory, Customer, Person, Store, SalesTerritory e SalesPerson.

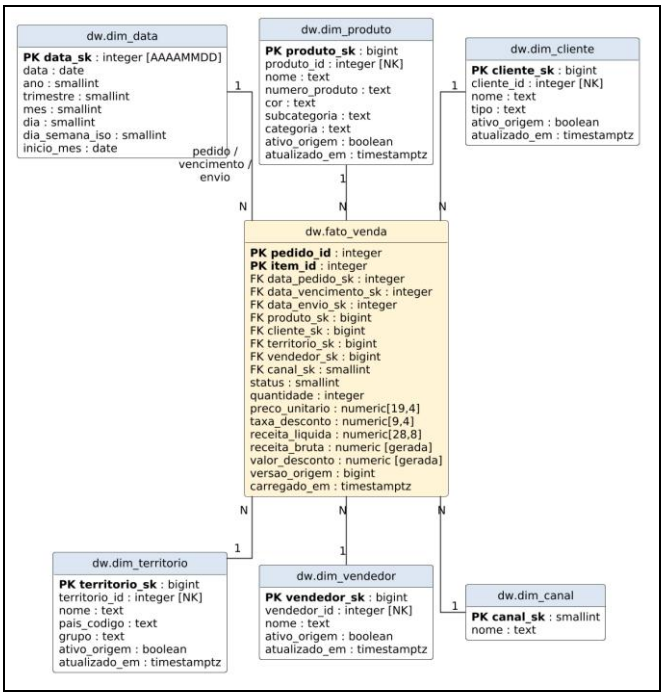
No OLTP, saber “a receita por categoria e território” exige unir ao menos seis tabelas; o nome do cliente vem de Person (pessoa física) ou de Store (loja); o canal é apenas uma *flag* (OnlineOrderFlag); e pedidos online não têm vendedor. Esses pontos orientaram a modelagem.

2.2 MODELO DE DADOS

2.2.1 Modelo multidimensional proposto

O modelo proposto segue o padrão estrela: uma tabela fato central, dw.fato_venda, ligada diretamente a seis dimensões — dim_data, dim_produto, dim_cliente, dim_territorio, dim_vendedor e dim_canal —, sem relações entre dimensões (Figura 1). O script de criação está em sql/dw.sql no repositório.

Figura 1 – Modelo estrela do Data Warehouse



Fonte: elaboração própria (2026), gerado em PlantUML.

2.2.2 Granularidade da tabela fato

O grão da fato_venda é **um item de pedido de venda**, ou seja, uma linha de SalesOrderDetail. Esse é o nível mais detalhado disponível e o único em que o produto é conhecido, já que um pedido pode conter vários produtos. Os atributos do cabeçalho (datas, cliente, território, vendedor e canal) se repetem em cada item do mesmo pedido. O número do pedido (pedido_id) é mantido na fato como dimensão degenerada e, junto com item_id, forma a chave primária, o que impede duplicações quando a carga é reexecutada.

As medidas aditivas são quantidade, receita_bruta, valor_desconto e receita_liquida; preco_unitario e taxa_desconto são não aditivas e servem a cálculos ponderados. A receita bruta e o desconto são colunas geradas pelo próprio PostgreSQL a partir da quantidade, do preço e da receita líquida, garantindo consistência entre as medidas.

2.2.3 Justificativa técnica das decisões de modelagem

Desnormalização das hierarquias. Subcategoria e categoria foram incorporadas à dim_produto, e os dados de Person e Store à dim_cliente. Isso evita o modelo floco de neve e mantém cada dimensão a uma junção da fato, como recomendam Kimball e Ross (2013).

Chaves substitutas. Cada dimensão possui chave própria do DW (*_sk) e preserva a chave natural da origem (*_id) com restrição de unicidade, isolando o DW de mudanças nas chaves do OLTP.

Dimensão de data com múltiplos papéis. Uma única dim_data atende às datas de pedido, vencimento e envio, o que permite medir prazos de expedição sem duplicar o calendário.

Membros “não informado”. Pedidos online não têm vendedor e alguns podem não ter território. Em vez de chaves nulas, foram criados membros com chave 0 (“Sem vendedor associado” e “Não informado”), mantendo as chaves estrangeiras obrigatórias e as junções internas corretas.

Dimensão de mudança lenta Tipo 1. As dimensões seguem o tipo 1 de *Slowly Changing Dimension* (SCD), isto é, são atualizadas por sobrescrita, pois os indicadores definidos não exigem histórico de atributos. Registros excluídos na origem são marcados com ativo_origem = false, sem apagar a linha, para não quebrar as referências da fato.

Integridade e desempenho. A fato possui chaves estrangeiras para todas as dimensões, restrições CHECK nas medidas (por exemplo, receita líquida igual a quantidade × preço × (1 − desconto)) e índices nas chaves usadas em filtros.

2.2.4 Dicionário de dados

O Quadro 1 descreve a tabela fato e o Quadro 2 resume as dimensões. O dicionário completo, com a origem de cada coluna, está em docs/dicionario_dados.md no repositório.

Quadro 1 – Dicionário de dados da tabela dw.fato_venda

Coluna	Tipo	Descrição
pedido_id, item_id	integer	Chave primária composta; pedido_id é dimensão degenerada
data_pedido_sk, data_vencimento_sk, data_envio_sk	integer	Chaves para dim_data nos papéis de pedido, vencimento e envio (envio pode ser nulo)
produto_sk, cliente_sk, territorio_sk, vendedor_sk, canal_sk	bigint/smallint	Chaves substitutas para as demais dimensões
status	smallint	Situação do pedido (1 a 6; 5 = expedido)
quantidade	integer	Unidades vendidas no item
preco_unitario, taxa_desconto	numeric	Preço de venda e desconto unitário (0 a 1)
receita_liquida	numeric(28,8)	Receita do item após desconto (LineTotal)
receita_bruta, valor_desconto	numeric (gerada)	quantidade × preço; receita bruta − receita líquida
versao_origem, carregado_em	bigint, timestamptz	Versão do Change Tracking e data da última gravação

Fonte: elaboração própria (2026).

Quadro 2 – Resumo das dimensões

Dimensão	Principais atributos	Linhas
dim_data	data_sk (AAAAMMDD), data, ano, trimestre, mês, dia, dia da semana, início do mês	1.139
dim_produto	produto_id, nome, número, cor, subcategoria, categoria	504
dim_cliente	cliente_id, nome (pessoa ou loja), tipo	19.820
dim_territorio	territorio_id, nome, país, grupo geográfico	11
dim_vendedor	vendedor_id, nome	18
dim_canal	canal_sk (0 = Revendedor, 1 = Online), nome	2

Fonte: elaboração própria (2026). Nota: produto, cliente, território e vendedor possuem ainda ativo_origem e atualizado_em.

2.3 DATA ETL

2.3.1 Estratégia de ETL incremental

A carga incremental usa o *Change Tracking* (CT) do SQL Server, que registra quais chaves foram inseridas, alteradas ou excluídas em cada tabela, com um número de versão crescente (Microsoft, 2026b). Essa abordagem foi preferida à comparação por ModifiedDate porque também identifica exclusões e não depende de a aplicação atualizar essa coluna. O script sql/origem.sql habilita o CT, com retenção de sete dias, nas dez tabelas de origem, além do isolamento *snapshot*.

O fluxo funciona assim: (a) na **primeira execução**, todos os dados são lidos e a versão atual do CT é gravada como *checkpoint* na tabela etl.controle; (b) nas **execuções seguintes**, a função CHANGETABLE(CHANGES tabela, checkpoint) devolve apenas as chaves alteradas desde a última carga; (c) mudanças indiretas são propagadas — a alteração de uma categoria, por exemplo, atualiza todos os produtos dela, e a de uma pessoa atualiza o cliente e o vendedor correspondentes; (d) todos os itens dos pedidos afetados são relidos, o que permite remover da fato os itens excluídos na origem; (e) dimensões e fatos são gravados por *upsert* (INSERT ... ON CONFLICT DO UPDATE), atualizando apenas linhas com valores realmente diferentes (IS DISTINCT FROM).

A leitura da origem ocorre em uma única transação *snapshot*, consistente com a versão lida. No destino, dimensões, fatos, registro de auditoria (etl.execucao) e novo *checkpoint* são confirmados na **mesma transação** PostgreSQL: se qualquer etapa falhar, nada é gravado e o *checkpoint* não avança. Uma trava (pg_try_advisory_lock) impede cargas simultâneas, e um *checkpoint* expirado ou uma origem restaurada exigem ressincronização explícita (run --resync). O Quadro 3 mostra os trechos centrais.

Quadro 3 – Trechos da extração incremental e da carga da fato

```
-- origem.py: chaves alteradas desde o checkpoint (uma consulta por tabela)
SELECT SalesOrderID, SalesOrderDetailID INTO #item
FROM CHANGETABLE(CHANGES Sales.SalesOrderDetail, @checkpoint) ct;
-- pedidos afetados = alterados no cabeçalho ou nos itens
SELECT SalesOrderID INTO #pedidos FROM #pedido UNION SELECT SalesOrderID FROM
#item;

-- etl.py: remove itens excluídos na origem, só nos pedidos afetados
DELETE FROM dw.fato_venda f WHERE f.pedido_id = ANY(%s) AND NOT EXISTS
  (SELECT 1 FROM lote_venda l WHERE l.pedido_id=f.pedido_id AND l.item_id=f.item_id);
-- upsert: grava apenas linhas novas ou com valores diferentes
INSERT INTO dw.fato_venda AS f (...) SELECT ... FROM lote_venda
ON CONFLICT (pedido_id, item_id) DO UPDATE SET quantidade=EXCLUDED.quantidade, ...
WHERE f.quantidade IS DISTINCT FROM EXCLUDED.quantidade OR ...;
-- mesma transação: checkpoint avança junto com os dados
INSERT INTO etl.controle(pipeline, versao, identidade_origem, execucao_id)
VALUES ('vendas', %s, %s, %s) ON CONFLICT (pipeline) DO UPDATE SET
versao=EXCLUDED.versao, ...;
```

Fonte: elaboração própria (2026), adaptado do repositório do projeto.

2.3.2 Fluxo da pipeline no Apache Airflow

No Airflow, a pipeline é definida como um *Directed Acyclic Graph* (DAG), isto é, um grafo acíclico dirigido de tarefas. A DAG adventureworks_simples (arquivo dags/adventureworks_dw.py) foi escrita com a interface de programação de aplicações (*Application Programming Interface* – API) *TaskFlow* do Airflow 3 (Apache Software Foundation, 2026). Ela é disparada manualmente (schedule=None) no ambiente de laboratório, permite apenas uma execução ativa por vez (max_active_runs=1), tenta novamente cada tarefa uma vez após 30 segundos e tem limite de 15 minutos por tarefa. As credenciais não ficam no código: as tarefas leem as *Connections* adventureworks_source (SQL Server) e adventureworks_dw (PostgreSQL) cadastradas no Airflow. O Quadro 4 descreve as tarefas.

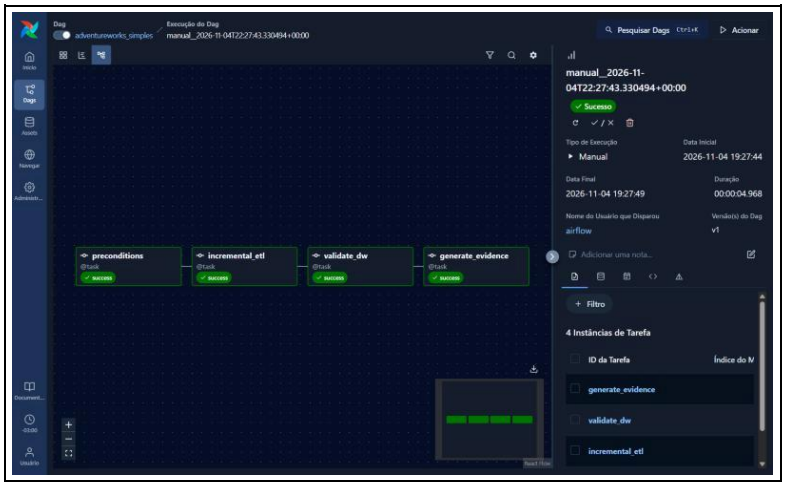
Quadro 4 – Tarefas da DAG adventureworks_simples

Tarefa	Função	Saída (XCom)
preconditions	Verifica, somente leitura, se o DW está criado, se o CT está ativo e se o checkpoint ainda é válido	ID da execução (*Universally Unique Identifier* – UUID derivado do run_id)
incremental_etl	Extrai as mudanças, carrega dimensões, calendário e fato e avança o checkpoint em uma transação	ID da execução
validate_dw	Confere se a execução corresponde ao checkpoint e se itens do mesmo pedido têm cabeçalho e datas coerentes	ID da execução
generate_evidence	Lê as dez views de indicadores no mesmo snapshot e grava um arquivo *JavaScript Object Notation* (JSON) de evidência	Caminho do arquivo

Fonte: elaboração própria (2026).

As dependências são lineares — preconditions >> incremental_etl >> validate_dw >> generate_evidence — e cada tarefa recebe da anterior o identificador da execução. Esse identificador é derivado do run_id do Airflow, portanto uma nova tentativa da mesma execução reconhece uma carga já confirmada e não a repete. A Figura 2 mostra o grafo da DAG em uma execução incremental concluída com sucesso, em cerca de 5 segundos.

Figura 2 – Grafo da DAG adventureworks_simples em uma execução incremental



Fonte: captura de tela do Apache Airflow, elaboração própria (2026).

2.3.3 Comprovação da carga incremental

Para comprovar a carga incremental, a quantidade do item 1 do pedido 43659 foi alterada de 1 para 2 e a cor do produto 776, de *Black* para *Blue*; a DAG foi executada, os valores originais foram restaurados e a DAG foi executada de novo. A Tabela 1 resume a auditoria (etl.execucao).

Tabela 1 – Execuções registradas em etl.execucao

Execução	Modo	Versão CT	Itens lidos	Itens gravados	Produtos gravados
Carga inicial	initial	– → 0	121.317	121.317	504
Sem alterações (2×)	incremental	0 → 0	0	0	0
Após alteração	incremental	0 → 2	12	1	1
Após reversão	incremental	2 → 4	12	1	1

Fonte: elaboração própria (2026), a partir da tabela etl.execucao.

Após a alteração, o ETL leu só os 12 itens do pedido afetado (de 121.317) e regravou apenas o item e o produto modificados; após a reversão, os indicadores voltaram aos valores originais.

2.4 INDICADORES

Os dez indicadores do Quadro 5 foram escolhidos para cobrir faturamento, volume, clientes, evolução temporal e nível de serviço. Todos são calculados sobre a *view*

dw.v_vendas, que considera apenas pedidos expedidos (status = 5) e já traz as datas de pedido, vencimento e envio. Os valores são comerciais, sem impostos e frete, e “receita líquida” significa receita após descontos, não lucro.

Quadro 5 – Indicadores definidos

Nº	Indicador	Cálculo	View
1	Receita bruta	Σ quantidade × preço unitário	kpi_01_receita_bruta
2	Descontos concedidos	Σ (receita bruta − receita líquida)	kpi_02_descontos
3	Receita líquida	Σ receita após desconto	kpi_03_receita_liquida
4	Unidades vendidas	Σ quantidade	kpi_04_unidades
5	Pedidos expedidos	contagem distinta de pedidos	kpi_05_pedidos
6	Ticket médio	receita líquida ÷ pedidos	kpi_06_ticket_medio
7	Taxa de desconto ponderada	descontos ÷ receita bruta × 100	kpi_07_taxa_desconto
8	Clientes com compra	contagem distinta de clientes	kpi_08_clientes_ativos
9	Crescimento mensal	(receita do mês − mês anterior) ÷ mês anterior × 100	kpi_09_crescimento_mensal
10	Expedição no prazo	pedidos enviados até o vencimento ÷ pedidos com data de envio × 100	kpi_10_expedicao_no_prazo

Fonte: elaboração própria (2026).

As divisões usam NULLIF para evitar divisão por zero, a taxa de desconto é ponderada pela receita e o crescimento mensal gera todos os meses do período, para que meses sem venda apareçam com receita zero. O Quadro 6 apresenta, como exemplo do script sql/kpis.sql, o cálculo do crescimento mensal.

Quadro 6 – Script SQL do indicador de crescimento mensal

```
CREATE OR REPLACE VIEW dw.kpi_09_crescimento_mensal AS
WITH intervalo AS (SELECT date_trunc('month',MIN(d.data)) inicio,
date_trunc('month',MAX(d.data)) fim
                FROM dw.fato_venda f JOIN dw.dim_data d ON d.data_sk=f.data_pedido_sk),
calendario AS (SELECT m::date mes FROM intervalo, generate_series(inicio,fim,interval '1 month')
m),
vendas AS (SELECT inicio_mes mes, SUM(receita_liquida) receita FROM dw.v_vendas GROUP
BY inicio_mes),
mensal AS (SELECT c.mes, COALESCE(v.receita,0) receita FROM calendario c LEFT JOIN
vendas v USING(mes)),
comparacao AS (SELECT mes, receita, LAG(receita) OVER (ORDER BY mes) receita_anterior
FROM mensal)
SELECT mes, receita, receita_anterior,
      100.0*(receita-receita_anterior)/NULLIF(receita_anterior,0) crescimento_pct FROM
comparacao;
```

Fonte: elaboração própria (2026), adaptado do repositório do projeto.

2.5 RESULTADOS DOS INDICADORES

A Tabela 2 apresenta os resultados obtidos após a carga, que cobre pedidos de 31 de maio de 2011 a 30 de junho de 2014. Os valores monetários estão em dólares americanos, moeda do AdventureWorks.

Tabela 2 – Resultados dos indicadores

Nº	Indicador	Resultado
1	Receita bruta	US$ 110.373.889,31
2	Descontos concedidos	US$ 527.507,91
3	Receita líquida	US$ 109.846.381,40
4	Unidades vendidas	274.914
5	Pedidos expedidos	31.465
6	Ticket médio	US$ 3.491,07
7	Taxa de desconto ponderada	0,48%
8	Clientes com compra	19.119 (96,5% dos 19.820 cadastrados)
9	Crescimento mensal	de −99,09% a +813,96% (38 meses)
10	Expedição no prazo	100% (31.465 pedidos)

Fonte: elaboração própria (2026), a partir das views do esquema dw.

A análise dos resultados, combinada a recortes pelas dimensões, revela alguns padrões nos dados:

a) **Concentração em bicicletas.** A categoria *Bikes* responde por 86,2% da receita líquida, seguida por *Components* (10,7%), *Clothing* (1,9%) e *Accessories* (1,2%). Em unidades, porém, roupas e acessórios somam 135.602 itens, quase metade do volume, o que mostra produtos de baixo valor e alta rotatividade.

b) **Dois canais com perfis opostos.** O canal de revendedores gerou US$ 80,5 milhões (73,3% da receita) em apenas 3.806 pedidos, com ticket médio de US$ 21.147,58; o canal online gerou US$ 29,4 milhões em 27.659 pedidos, com ticket médio de US$ 1.061,45. O ticket médio geral (US$ 3.491,07) esconde essa diferença, e por isso o indicador deve ser analisado por canal.

c) **Oscilação mensal causada pelos revendedores.** O crescimento mensal alterna fortes altas e quedas (+813,96% em outubro de 2011; −75,10% em abril de 2014) porque os pedidos de revenda se concentram em alguns meses: junho, setembro e novembro de 2011 não tiveram nenhum, e fevereiro e abril de 2014 quase nada. Já o canal online cresce de forma estável, passando de cerca de US$ 0,5 milhão por mês em 2012 para US$ 1,9 milhão em maio de 2014. A queda de −99,09% em junho de 2014 reflete apenas o fim do período de dados.

d) **Crescimento anual e mudança no perfil de pedidos.** A receita passou de US$ 33,5 milhões em 2012 para US$ 43,6 milhões em 2013 (+30,1%), enquanto o número de pedidos saltou de 3.915 para 14.182. O aumento veio sobretudo do canal online, que passou de 2.743 para 12.584 pedidos, com pedidos de menor valor.

e) **Regularidade típica de dados sintéticos.** A taxa de desconto é baixa (0,48%) e todos os pedidos foram enviados sete dias após o pedido, com vencimento em doze dias, o que leva a 100% de expedição no prazo. Esse resultado reflete a natureza fictícia do AdventureWorks.

3 CONSIDERAÇÕES FINAIS

O trabalho atingiu o objetivo de construir um *Data Warehouse* de vendas a partir do AdventureWorks2016. O modelo estrela simplificou as consultas, e os dez indicadores

foram calculados com *views* SQL curtas. A pipeline no Apache Airflow organiza o processo em quatro tarefas com dependências claras, e a carga incremental baseada em *Change Tracking* mostrou-se eficaz: uma alteração em um único item levou o ETL a processar apenas o pedido afetado, sem recarregar os demais 31.464 pedidos. Os indicadores revelaram padrões relevantes, como a dependência da categoria *Bikes*, a diferença de perfil entre os canais e a oscilação mensal causada pelos revendedores, mostrando que os resultados precisam ser lidos pelas dimensões e não apenas pelos totais.

Como limitações, as dimensões (SCD Tipo 1) não guardam histórico, a retenção do *Change Tracking* é de sete dias e a DAG é disparada manualmente. Como trabalhos futuros, sugerem-se agendar a DAG diariamente, adotar SCD Tipo 2 em cliente e produto, calcular a margem de lucro com o custo padrão dos produtos e incluir outros processos, como compras e estoque.

REFERÊNCIAS

ADVENTUREWORKS-DW: data warehouse AdventureWorks com Apache Airflow. [S. l.]: GitHub, 2026. Repositório de código-fonte. Disponível em: https://github.com/lucaslnasc/adventureworks-dw. Acesso em: 4 nov. 2026.

APACHE SOFTWARE FOUNDATION. **Apache Airflow documentation**: version 3.3.2. [S. l.], 2026. Disponível em: https://airflow.apache.org/docs/apache-airflow/3.3.2/. Acesso em: 4 nov. 2026.

CENTRO UNIVERSITÁRIO SALESIANO. **Guia de elaboração e normalização de trabalhos acadêmicos e de pesquisa**. Vitória, 2024. Disponível em: https://unisales.br/wp-content/uploads/2024/07/NOVO-GUIA-DE-ELABORACAO-E-NORMALIZACAO-DE-TRABALHOS-ACADEMICOS-E-DE-PESQUISA-29.05.pdf. Acesso em: 4 nov. 2026.

INMON, William H. **Building the data warehouse**. 4. ed. Indianapolis: Wiley, 2005.

KIMBALL, Ralph; ROSS, Margy. **The data warehouse toolkit**: the definitive guide to dimensional modeling. 3. ed. Indianapolis: Wiley, 2013.

MICROSOFT. **Bancos de dados de exemplo AdventureWorks**. [S. l.], 2026a. Disponível em: https://learn.microsoft.com/pt-br/sql/samples/adventureworks-install-configure. Acesso em: 4 nov. 2026.

MICROSOFT. **Sobre o controle de alterações (SQL Server)**. [S. l.], 2026b. Disponível em: https://learn.microsoft.com/pt-br/sql/relational-databases/track-changes/about-change-tracking-sql-server. Acesso em: 4 nov. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 17 documentation**. [S. l.], 2026. Disponível em: https://www.postgresql.org/docs/17/. Acesso em: 4 nov. 2026.

STAR schema. In: WIKIPEDIA: the free encyclopedia. [S. l.], 2026. Disponível em: https://en.wikipedia.org/wiki/Star_schema. Acesso em: 4 nov. 2026.